

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Sergiu Gherasim, FAF-242

Report

Laboratory Work No. 2

Operating Systems

Topic: Meet the OS You Will Build

Building, Using and Extending xv6 on RISC-V

Checked by:
Patricia Reiţman, *university assistant*

Chişinău – 2026

CONTENTS

Introduction **3**

1 Building and Booting xv6 **4**

1.1 The Build 4

2 Using xv6 as a Unix **5**

2.1 Pipes 6

3 Reading the Source **8**

3.1 A User Program 8

3.2 The System Call Interface 10

4 Adding a User Program **12**

4.1 Using the Sleep System Call 12

4.2 Teaching the Build About It 13

4.3 Testing Inside xv6 14

Conclusions **16**

INTRODUCTION

In Laboratory Work No. 1 the operating system was met from the outside: a finished Linux system was observed through the terminal, and the programs could only be seen as black boxes that ask the kernel for help. In this laboratory work the view is reversed. The system itself is built from source, so both sides of every request become readable: the user program that asks and the kernel code that answers.

The system is **xv6** for RISC-V, a modern re-implementation of Unix Version 6 written in about 9 000 lines of C. It is a complete, bootable Unix, but small enough to read the whole kernel. It is compiled with a RISC-V cross-compiler and started under QEMU, an emulator that pretends to be a RISC-V computer. The work has four steps: clone and build xv6, use it as a tiny Unix, read the source of a user program and of the system calls behind it, and finally add a new program, `sleep`, to the system and to its build.

I worked on the Linux virtual machine of the course, which already contains the toolchain. The submitted files are `user/sleep.c` and the modified `Makefile`; the answers to the two *Observe* boxes are in `notes.md`.

```
sergiu@debian:~$ cd ~
riscv64-linux-gnu-gcc --version | head -1
qemu-system-riscv64 --version | head -1
git clone https://github.com/mit-pdos/xv6-riscv.git
cd xv6-riscv
ls
bash: riscv64-linux-gnu-gcc: command not found
QEMU emulator version 10.0.13 (Debian 1:10.0.13+ds-0+deb13u1)
fatal: destination path 'xv6-riscv' already exists and is not an empty directory.
fs.img  kernel  LICENSE  Makefile  mkfs  README  test-xv6.py  user
sergiu@debian:~/xv6-riscv$
```

Figure 1: The RISC-V toolchain, QEMU and the cloned xv6 tree

The cross-compiler is a normal GCC, but it produces code for another processor: `riscv64-linux-gnu-gcc` runs on the Linux VM and writes RISC-V binaries, so the kernel can be built without a RISC-V computer. `qemu-system-riscv64` is the emulator that will execute them. After `git clone` the tree contains the kernel in `kernel/`, the programs in `user/`, the `Makefile` that builds both, and the `README`.

1 BUILDING AND BOOTING XV6

1.1 The Build

`make qemu` does two things. First it builds `kernel/kernel`, the operating system itself, and every program listed in the `UPROGS` variable of the `Makefile`. Then a tool called `mkfs` packs those programs, together with `README` and the device files, into `fs.img`, a small disk image that xv6 will find as a file system. Finally `make` starts QEMU with the kernel, the disk image and no graphics window, so the machine appears as text in the terminal of the VM.

```
sergiu@debian:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=fals
e -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35560
forktest       2 5 17464
grep           2 6 40176
init           2 7 36016
kill           2 8 35496
ln             2 9 35312
ls             2 10 38856
mkdir          2 11 35544
rm             2 12 35528
sh             2 13 58608
stressfs       2 14 36416
usertests      2 15 202984
grind          2 16 51792
wc             2 17 37632
zombie         2 18 34896
logstress      2 19 37480
forphan        2 20 36344
dorphan        2 21 35808
sync           2 22 34960
sleep          2 23 35280
args           2 24 35280
console        3 25 0
$
```

Figure 2: make qemu: end of the build, the xv6 boot banner and ls

The last lines of the build are the linking of the kernel. Then the emulated machine starts: xv6 prints `xv6 kernel is booting`, starts the first process, `init`, which prints `init: starting sh` and gives the shell. The prompt is `$`, and the `ls` that follows runs *inside* xv6, not on Linux: it lists the files of the xv6 disk image, which are the programs `mkfs` packed there.

To leave QEMU the sequence is `Ctrl-A` and then `X`. This is handled by QEMU itself, not by xv6; the small shell of xv6 has no exit command.

2 USING XV6 AS A UNIX

The programs of xv6 are the ancestors of the Linux commands used in Laboratory Work No. 1, with the same names and the same habits: they read from standard input, write to standard output and can be chained with a pipe.

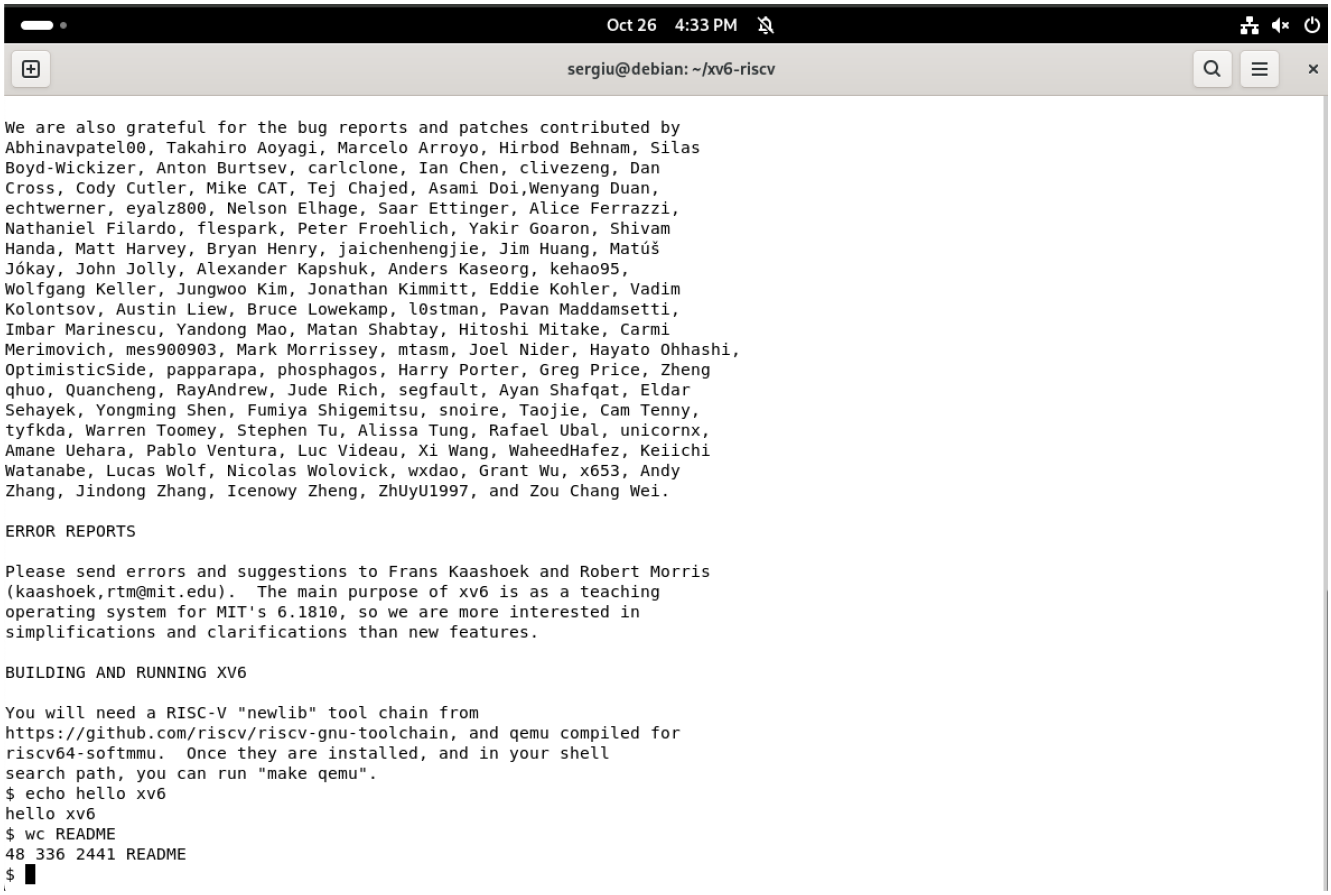


Figure 3: cat README, echo hello xv6 and wc README inside xv6

`cat README` prints the text of the file to the terminal: `cat` opens the file, copies it in blocks into a buffer and writes each block out. `echo hello xv6` prints its arguments; the same text sent through a pipe, `echo hello xv6 | wc`, would instead be counted. `wc README` prints three numbers, the lines, the words and the bytes of the file, followed by its name.

2.1 Pipes

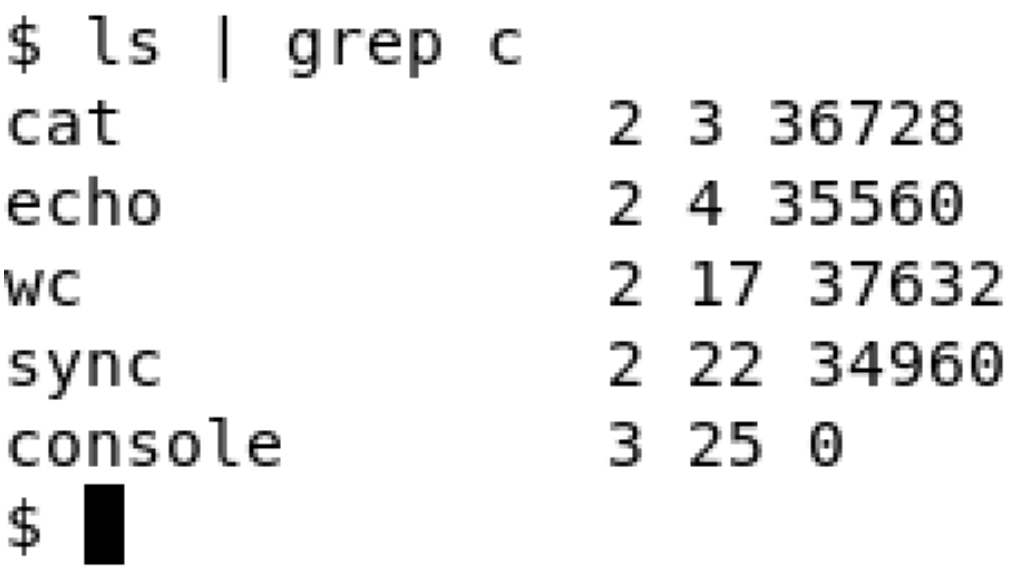


Figure 4: A pipe inside xv6: ls | grep c

A shell builds a pipe with the `pipe` system call, which returns two file descriptors: one writes, the other reads. Then it starts both programs with `fork`, and each child replaces itself with `exec`: the left program writes into the pipe, the right one reads from it. Here `ls` lists the programs of xv6 and `grep c` keeps only the names that contain the letter `c`. Neither program knows about the other; the kernel moves the bytes between them.

Program	What it does
`ls`	lists the entries of a directory
`cat`	copies files to standard output
`echo`	prints its arguments
`wc`	counts lines, words and bytes
`grep`	prints the lines that match a pattern
`usertests`	runs the kernel test suite (optional)

Table 1: Some programs that xv6 ships with, all of them under `user/`.

1. List three programs xv6 ships with.

`cat`, `echo` and `wc`. The full list is the `UPROGS` variable of the `Makefile`, and the source of each one is a single small file in `user/`.

2. You just used a pipe (`|`) inside xv6. Which two OS features must exist for a pipe to work?

Processes and **inter-process communication**. A pipe only makes sense between two processes running at the same time, so the kernel must be able to create (`fork`), run and schedule them. And because the two processes have separate address spaces, they cannot share a buffer directly: the kernel must own a communication channel between them and copy the bytes through it.

3. In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?

It is much smaller: the same basic commands and pipes work, but there are far fewer of them (for example `clear` does not exist), there is no command history or completion, and it is noticeably slower, because every program is started with a fresh `fork` and `exec` on an emulated processor.

3 READING THE SOURCE

The tree is split into two folders, and this split is the shape of every operating system. `kernel/` is the system itself: it runs in supervisor mode, keeps the page table, schedules the processes, stores the files and handles the system calls. `user/` holds ordinary programs that run on top of it. A program in `user/` cannot touch the disk or the devices; it can only ask the kernel with a system call.

3.1 A User Program

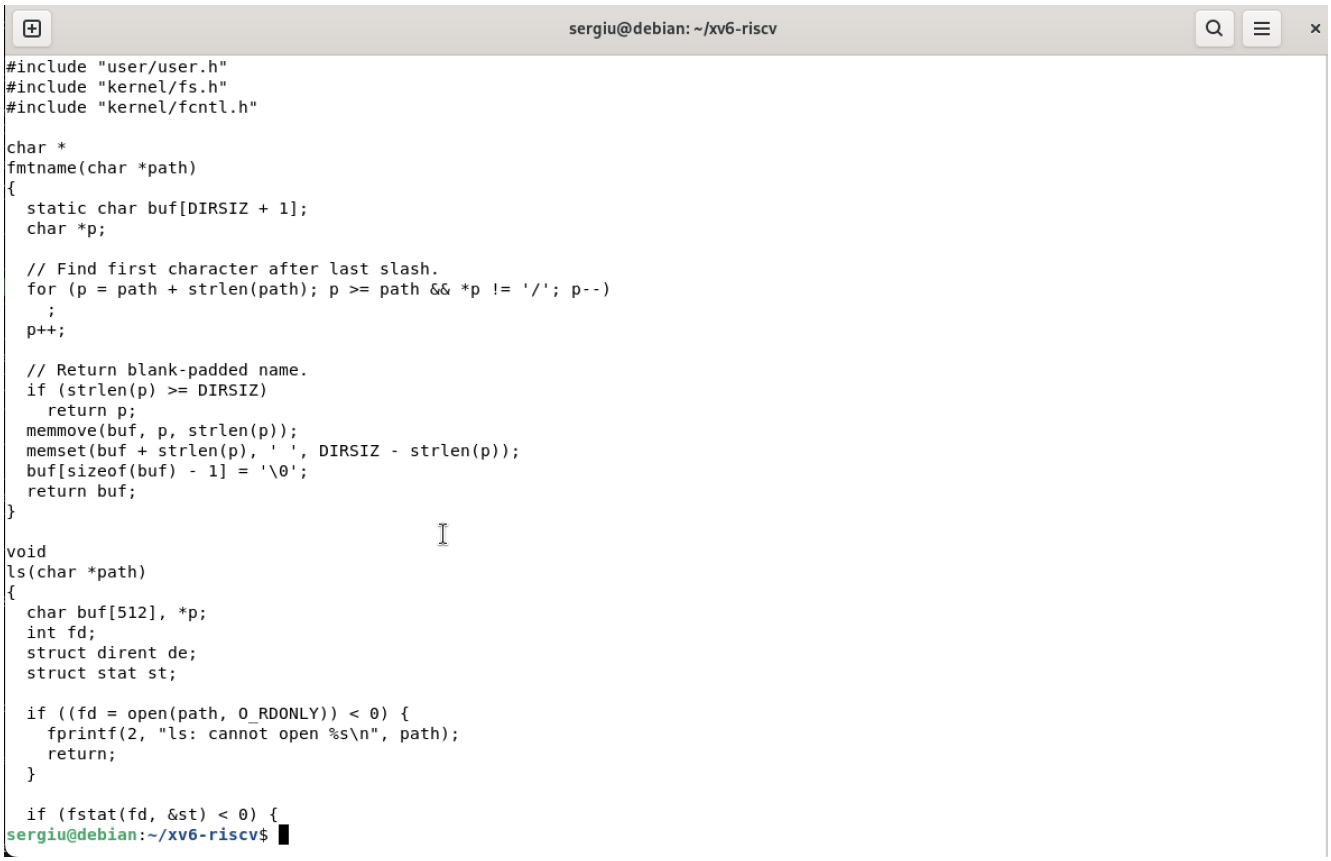


Figure 5: The first lines of user/ls.c

`user/ls.c` is a complete `ls` in about a hundred lines. The shape is the usual C program: `main` reads the arguments and calls `ls` for each path. `ls` then `open`s the path, `read`s the directory entries block by block and prints a name for every entry; for each one it calls `stat` to know the type and the size. The functions `fmtname` and `ls` are local, but `open`, `read`, `close` and `fstat` are not: they are declared in `user/user.h` and implemented in the kernel.

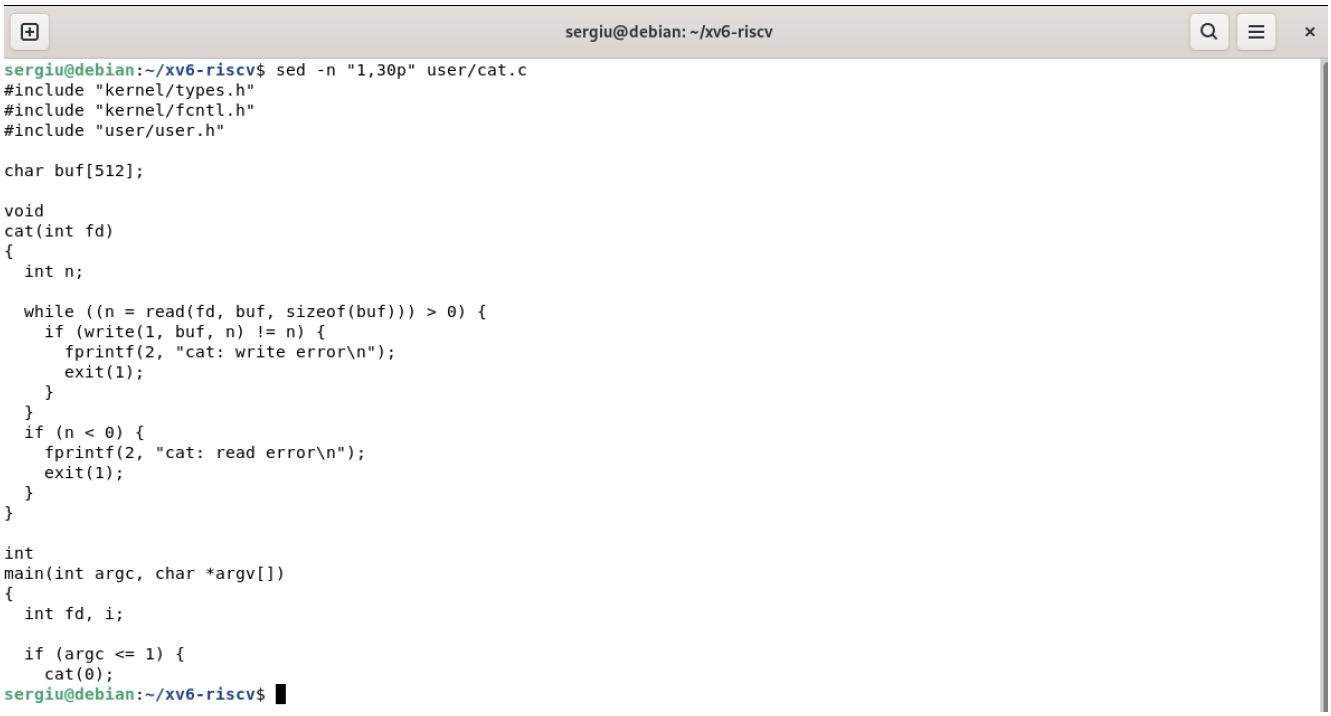


Figure 6: The first lines of user/cat.c

`user/cat.c` is even smaller. Its `main` calls `cat` for every file name, and `cat` does one thing in a loop: `read` a block into a buffer, `write` that buffer to descriptor 1, repeat until `read` returns 0. If a file cannot be opened, it prints an error on descriptor 2 and exits with status 1.

3.2 The System Call Interface

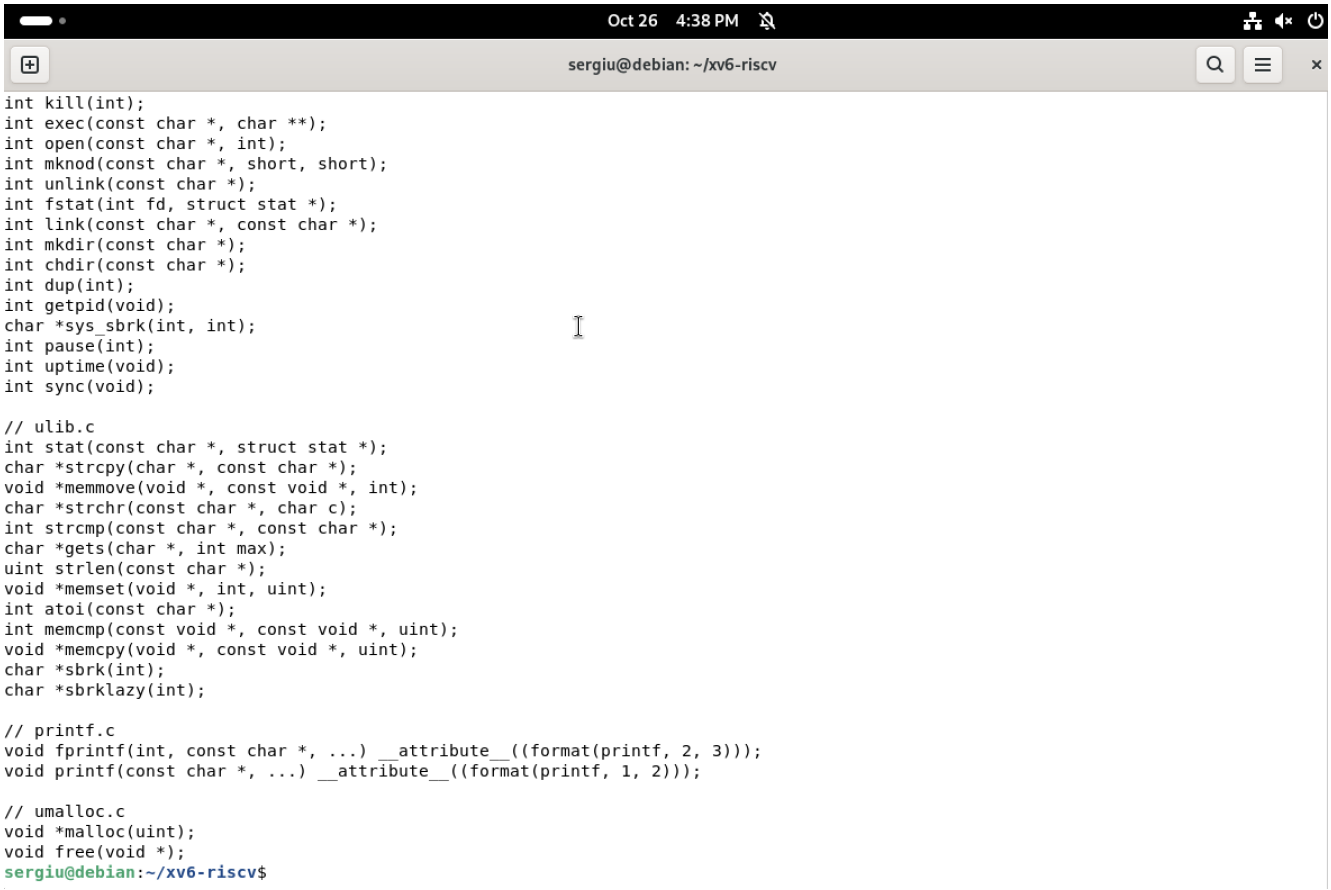


Figure 7: user/user.h, the list of system calls available to a program

`user/user.h` is the contract between the two folders. It declares a few library helpers (`printf`, `atoi`, `malloc`) and, below them, every system call a user program may make. These functions have no body in `user/`: their implementation is a few instructions of assembly (in `user/usys.S`) that put the call number in a register and execute the `ecall` instruction. That single instruction switches the processor into supervisor mode and jumps into the kernel, which does the real work and returns.

```
sergiu@debian:~/xv6-riscv$ grep -nE "sys_read|sys_write" kernel/sysfile.c
69:sys_read(void)
83:sys_write(void)
sergiu@debian:~/xv6-riscv$
```

Figure 8: Locating sys_read and sys_write in the kernel

On the other side of `ecall` sits `kernel/syscall.c`, which reads the call number and dispatches to the right function through the table `syscalls`. The file system calls themselves are in `kernel/sysfile.c`: `sys_read` first fetches the arguments from the user process with `argfd` and `argaddr`, then calls `fileread` on the open file, which finally asks the inode to read the blocks.

1. Which system calls does `user/cat.c` use, and what does each one ask the kernel to do?

It uses `open`, `read`, `write`, `close` and `exit`.

Call	Request to the kernel
`open`	find the named file, check the permissions and return a file descriptor
`read`	copy the next bytes of an open file into a buffer of the process
`write`	copy bytes from a buffer to an open file (here descriptor 1, the terminal)
`close`	release a file descriptor so the number may be used again
`exit`	terminate the process and give its status to the parent

Table 2: The system calls of `cat`. Each one is a request that only the kernel may perform.

2. In which kernel file, and on which line, is `sys_read` implemented?

In `kernel/sysfile.c`. The `grep -n` output shows `sys_read` at line 69, and the next function, `sys_write`, at line 83. The numbers are the ones of the current `xv6-riscv` master; they move when the file changes, which is why one searches for the name instead of remembering the line.

3. In one sentence: what is the difference between the code in `kernel/` and the code in `user/`?

`kernel/` is the operating system itself, which runs privileged, owns the hardware and cannot be removed without the machine stopping, while `user/` holds the ordinary programs that run unprivileged on top of it and may be replaced or deleted freely.

4 ADDING A USER PROGRAM

4.1 Using the Sleep System Call

The kernel already implements the call that pauses a process: it takes a number of clock ticks, marks the calling process as sleeping and calls the scheduler, which runs something else until the timer wakes the process up. In the current `xv6-riscv` this system call is named `pause`; it was called `sleep` until August 2025, when it was renamed to avoid confusion with the program of the same name (older copies of the tree, such as the one used by the task sheet, still show `sleep`). What is missing is a program that uses it. The task asks for a program that takes exactly one argument and, if the argument is missing, prints a usage message instead of crashing.

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // the sleep system call of current xv6
  exit(0);
}
```

Listing 1: user/sleep.c, the new program

`argc` counts the words of the command line including the program name, so a correct call has `argc == 2`. `argv[1]` is the text typed after the name, and `atoi` turns it into a number, because the system call needs an integer. `fprintf(2, ...)` writes the usage message on descriptor 2, standard error, and `exit(1)` reports failure to the shell. A successful run ends with `exit(0)`.

```
sergiu@debian:~/xv6-riscv$ vim user/sleep.c
sergiu@debian:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
sergiu@debian:~/xv6-riscv$
```

Figure 9: The new file user/sleep.c

4.2 Teaching the Build About It

A file in `user/` is not compiled by itself. The `Makefile` holds the list `UPROGS` of the programs that go into `fs.img`, and only those are built. Each line is the name of the object with an underscore in front, followed by a backslash, and the indentation is a tab: the list is one long command continued from line to line, so a space instead of a tab breaks it.

```
sergiu@debian:~/xv6-riscv$ sed -n '/^UPROGS=/,/^$/p' Makefile | tail -5
        $U/_dorphan\
        $U/_sync\
        $U/_sleep\
        $U/_args\

sergiu@debian:~/xv6-riscv$
```

Figure 10: UPROGS in the Makefile with the new _sleep entry

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        ...
        $U/_wc\
        $U/_zombie\
        $U/_sleep\
```

Listing 2: The end of the UPROGS list after the change

4.3 Testing Inside xv6

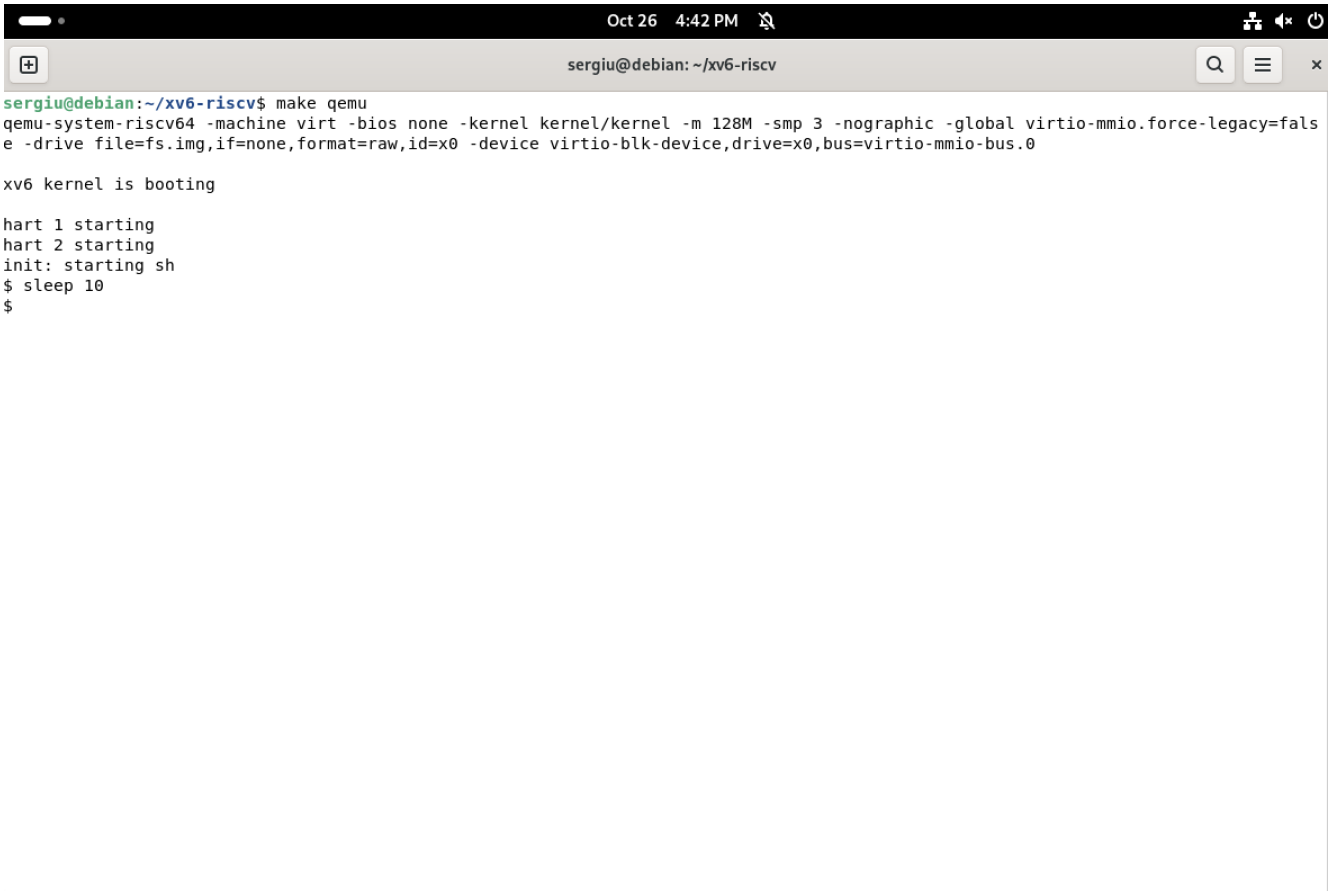


Figure 11: Rebuilding, then sleep 10 inside xv6

After `make qemu` the new program is part of the disk image, so it appears in `ls` like the others. `sleep 10` returns to the prompt after ten clock ticks without printing anything: the process really did stop, and the shell was not running meanwhile, which shows that the scheduler had other work to do or the processor was idle. This is the first program of the course that calls into a kernel compiled on the same machine.

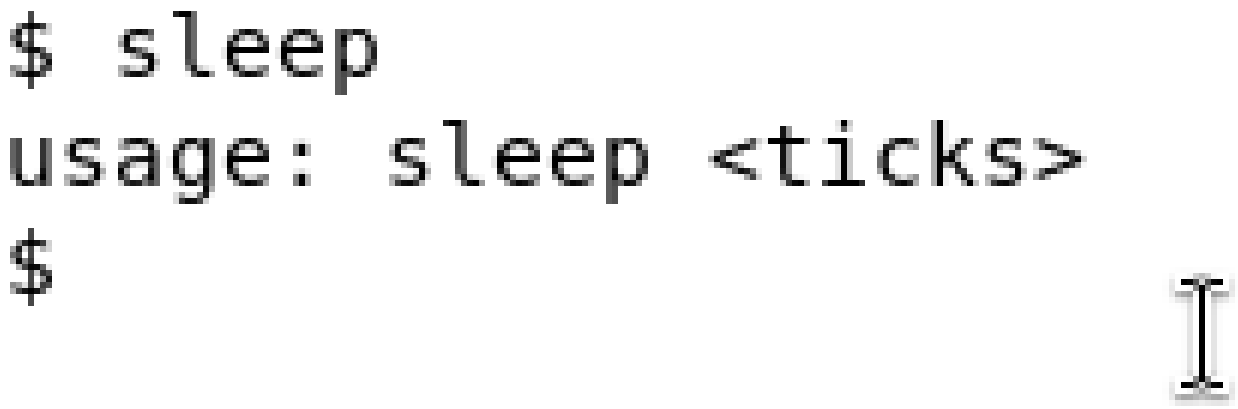


Figure 12: Calling sleep without an argument prints the usage message

With no argument, `argc` is 1 and the program takes the error branch: it prints `usage:`

`sleep <ticks>` and exits with status 1. The shell simply shows the next prompt, since xv6 does not report exit statuses by default.

CONCLUSIONS

In this laboratory work I built an operating system from source and ran it on emulated hardware. Comparing it with Laboratory Work No. 1 makes the position of the kernel clear: on Linux I could only see programs from the outside, with `ps` and `/proc`, while in xv6 I could read both sides of the same request. The user program in `user/` calls `read` or `sleep`, the assembly stub executes `ecall`, the kernel looks the call up in its table and runs `sys_read` or `sys_pause`; nothing in `user/` can touch a disk or a device by itself.

The pipe experiment showed the two features that make this cooperation useful: processes, created and scheduled by the kernel, and inter-process communication, a channel owned by the kernel because the processes cannot share memory directly. Writing `sleep` closed the circle: a new file in `user/`, one tab-indented line in `UPROGS`, and the program becomes part of the system that is built and booted. The kernel and the programs are the same kind of C code, separated only by privilege and by the system call interface.